Project Report

ML project 2024: Animal Classification

Soji John
0626734

January 15, 2025

1. INTRODUCTION

Image classification is amongst the most frequently implemented task in machine learning, where the objective is to assign an image to one of several predefined categories. In this project, the aim is to develop a robust classifier capable of accurately distinguishing between 12 distinct animal classes. The classifier leverages various image classification techniques combined with appropriate preprocessing steps to achieve optimal performance.

This project employs a traditional approach using Visual Bag-of-Words (VBoW) models for feature representation, implemented in Python through Jupyter Notebooks. Since labeled training data is used to train the model to classify samples into multiple categories, this task is a supervised multiclass classification problem.

The evaluation metric for this task is Cross-Entropy Loss, a standard measure in classification problems that evaluates how well the predicted probabilities match the true labels. The loss is defined as

$$L = -\frac{1}{N}\sum_{n=1}^{N}\sum_{c=1}^{C} y_n^c \log(p_n^c)$$

where *N* is the number of training/test samples, *C* is the number of classes and *p(n,c)* is the probability that sample *n* belongs to class *c* as output of the classifier. *y(n,c)* is the true class probability that sample *n* belongs to class *c*. Minimizing this Log Loss ensures not only accurate predictions but also high confidence in those predictions, as accuracy alone is insufficient to evaluate the model's reliability.

This project was conducted as part of a Machine Learning course and was also tested in a Kaggle competition setting, demonstrating its practical application. The following sections detail the methodology, results, and discussions surrounding the development and evaluation of this animal classification AI.

2. Data Analysis

The dataset comprises of the following 12 distinct animal classes:

- Chicken
- Elephant
- Fox
- German Shepherd
- Golden Retriever
- Horse
- Jaguar
- Lion
- Owl
- Parrot
- Swan, and
- Tiger

The ‘train’ folder has the training data consisting of labeled images, with labels derived from their respective sub-folder names. The ‘test’ folder is a single folder containing all the unlabeled test images of the various animals grouped together. This test folder is to be used

for testing the developed ML model, to evaluate the accuracy of our most suitable model or image classifier. Overall, we had the following count of images for training and testing.

- Number of training images: 4042
- Number of test images: 4035

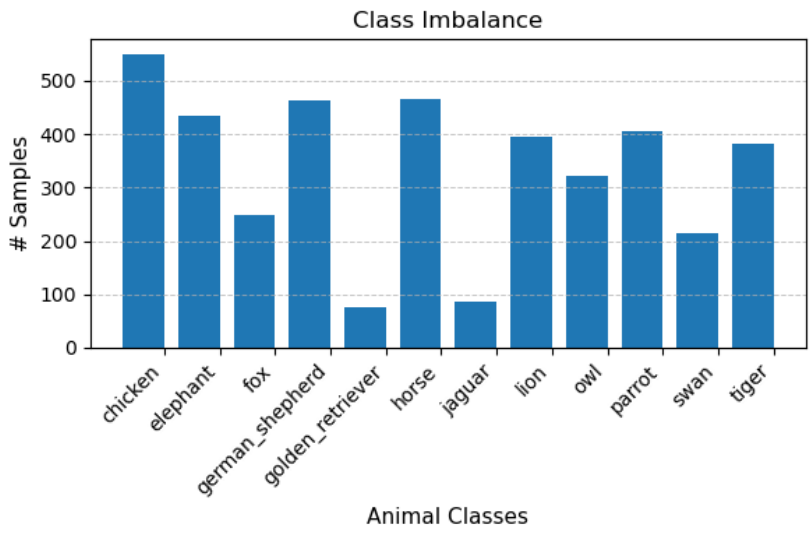
Next, we plotted the first 3 image files in each of the 12 folders representing the 12 animal classes as shown in the figure here

Class Imbalance:

The figure below shows the number of images per class. We can see that 7 of the 12 classes have 400 or more images, while 2 have less than 100 images, indicating class imbalance. Class imbalance in a training dataset can significantly impact the performance and behaviour of a ML model.

Key consequences of Class Imbalance:

- The model may become biased toward predicting the majority classes while the minority classes may have high misclassification rates because the model may not learn its patterns effectively
- Standard evaluation metrics like accuracy can be misleading
- The model might fail to generalize well to unseen data if the minority class is underrepresented



3. Preprocessing and feature extraction

In this stage of the project, images are pre-processed for reducing the noise of the current dataset and then, feature extraction techniques are applied. Preprocessing and feature extraction play a crucial role in ensuring the model's effectiveness and accuracy.

- **Image preprocessing** involves transforming raw image data into a format that is more suitable for analysis by machine learning models. This step is critical because raw images often contain noise, inconsistencies, and irrelevant information that can adversely impact model performance.
- **Feature extraction** is the process of converting images into a structured format that a machine learning model can process and analyze. In this project, a Visual Bag-of-Words (VBoW) approach is used for feature representation. VBoW identifies key patterns or "visual words" in the images by extracting local descriptors such as SIFT (Scale-Invariant Feature Transform) features. These features capture essential characteristics like edges, textures, and corners, which are critical for distinguishing between animal types

The synergy of preprocessing and feature extraction ensures that the training data is clean, consistent, and informative. This enables the supervised learning algorithm to focus on meaningful distinctions between animal classes, rather than being distracted by irrelevant noise or inconsistencies. Together, these steps form the foundation for building a classifier that is both accurate and generalizable.

Feature Descriptor

Features such as edges, textures, shapes, colors, etc, allow models to differentiate between categories. A feature descriptor (e.g., DAISY, ORB, SIFT) is like a "translator" used to transform raw image data into a structured representation that ML models can understand. In other words, it helps extract meaningful and representative features from an image, and this information is then encoded into feature vectors that summarize the image's content in a compact form.

The following experiment was carried out to with different descriptors to see how they affect classification accuracy.

1. Pre-computed image features for the given descriptor are loaded from a pickle file, which is nothing but file serialized for python. This avoids recalculating features for every run.
2. A codebook of size 500 is created using clustering and it serves as a vocabulary of "visual words" for VBoW encoding.
3. For each feature descriptor, the training data was fed into a Random Forest model. For simplicity, codebook size (refers to the number of "visual words" or clusters that are created during the feature encoding process) was kept 500 for the experiment.
4. The output provides the average Log Loss for various feature descriptors. Log Loss evaluates how well the predicted probabilities match the true labels, with lower values indicating better performance.

Feature Descriptors	Average Log Loss
boost_desc	1.955
daisy	1.757
lucid	2.064
orb	2.337
sift	2.049
vgg	1.869
freak	1.892

Observations:

Best Performing Descriptors:

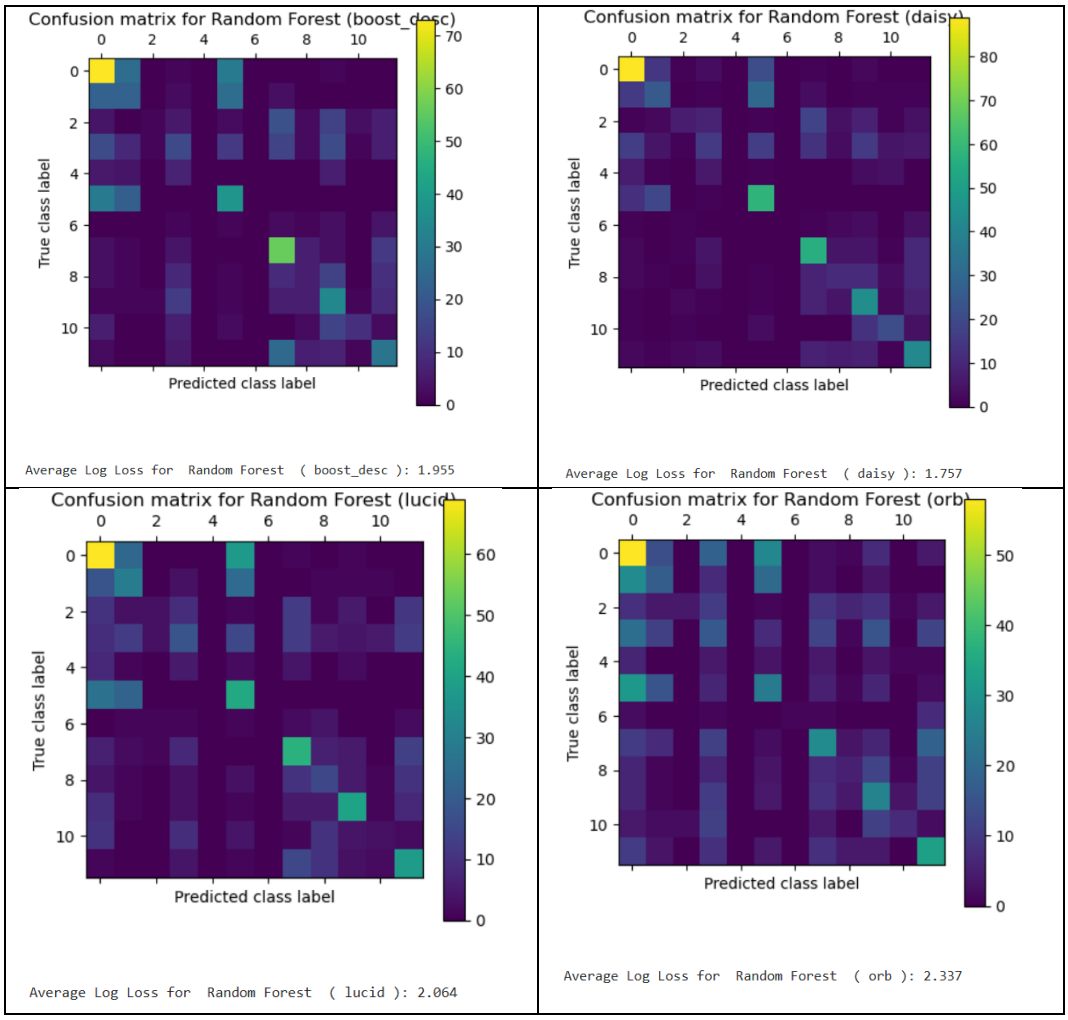
- **Daisy** has the lowest Log Loss (1.757), making it the most suitable feature descriptor for this project. It indicates that the Daisy descriptor provides the Random Forest model with the most discriminative and robust features for classifying the animal images.
- **Boost_Desc** follows closely with a Log Loss of 1.955, suggesting it is a strong alternative.

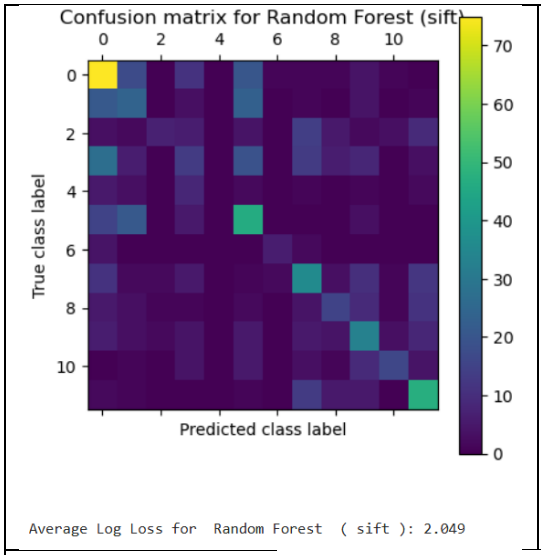
Descriptors with Moderate Performance:

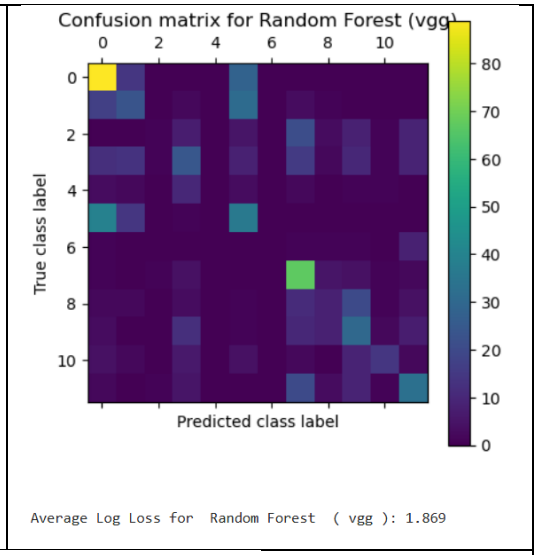
- **VGG (1.869)** and **Freak (1.892)** perform reasonably well. While not as effective as Daisy, they still provide relatively low Log Loss values compared to other descriptors.
- **SIFT (2.049)** is slightly less effective but may still be useful in combination with other descriptors.

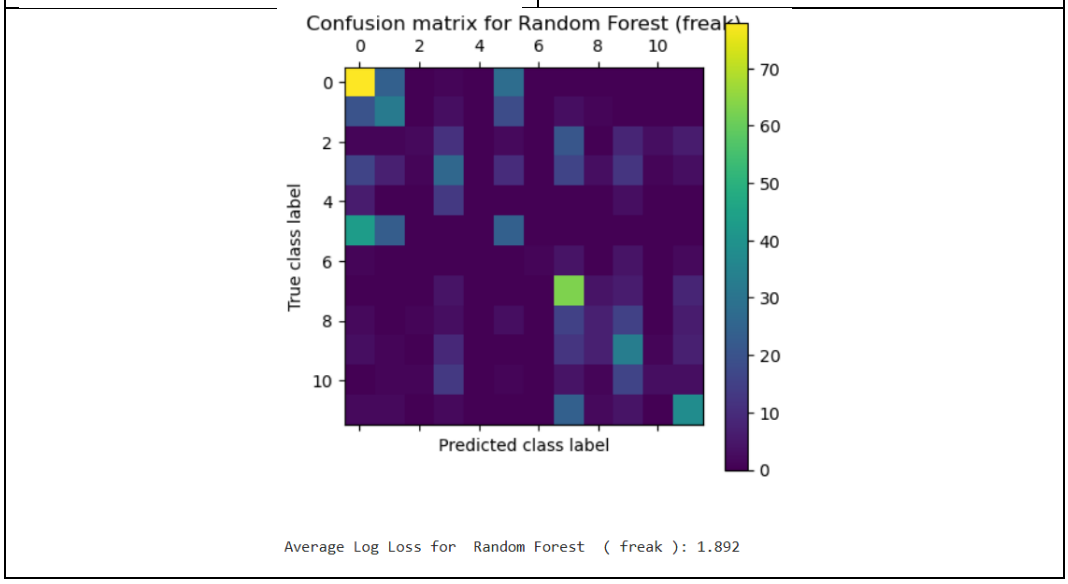
Least Performing Descriptors:

- **Lucid (2.064)** and **ORB (2.337)** exhibit the highest Log Loss values, indicating poor performance. These descriptors may not extract features that are adequately informative or discriminative for this dataset.









4. Training – Validation Test split

As per requirement, the test data cannot be used for model optimization. It is chosen to take into account the unbalance of the dataset given. So we split the training data a second time into a training set and a validation set. Our model is trained on the training dataset, whereas it's performance or accuracy is measured on the basis of how well the model could classify the images for the validation dataset

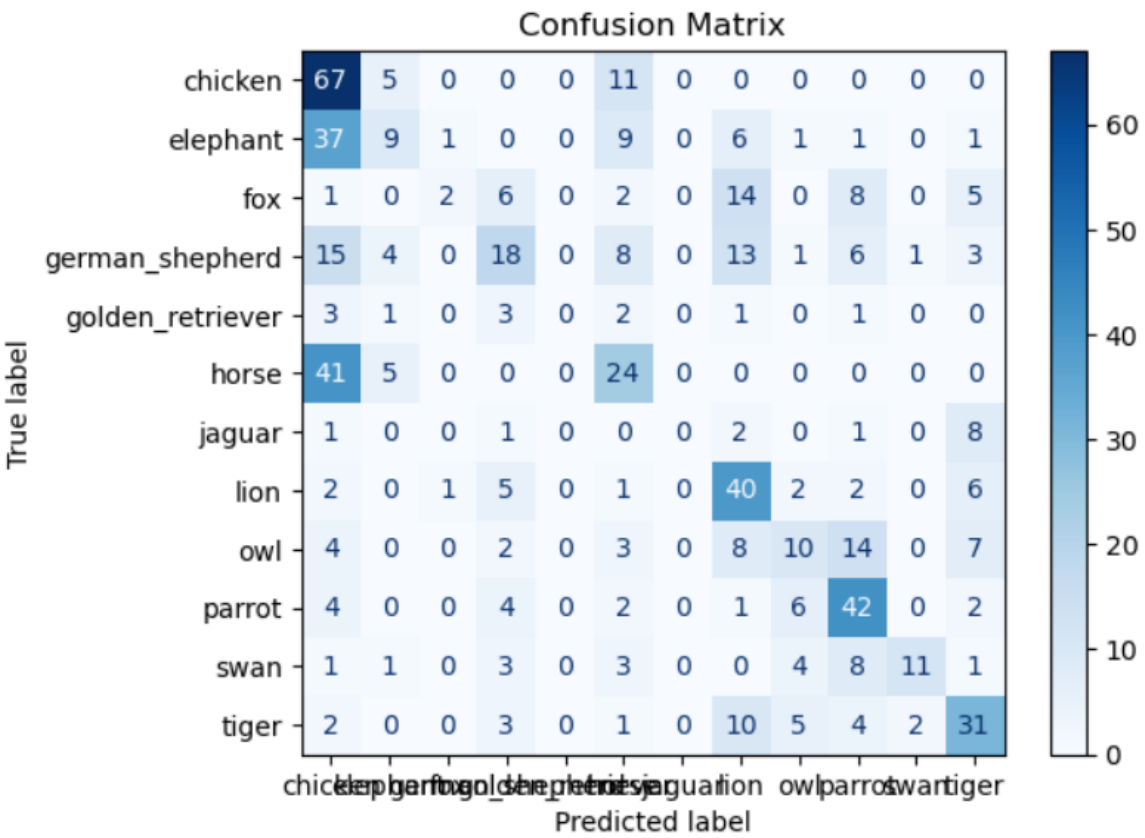
For the training-validation division, we opted for the standard 85% training and 15% validation split. Given the dataset size and the relatively sparse nature of the feature vectors (a common characteristic of bag-of-words models), this split is considered appropriate. Preprocessing, feature selection, and further fine-tuning will be applied solely to the 85% training portion to prevent any potential biases from information leakage.

5. Model training, Analysis and Conclusion

The following ML models were used and analyzed for performance.

1. **Random Forests:** This model builds multiple decision trees and combine their predictions. It is effective for both binary and multi-class classification, especially when you have a large and diverse set of features.
2. **Support Vector Machines (SVM)**: SVM uses some extraordinary complex data transformations based on which it then finds an optimal boundary between the possible outcomes. SVM works well in these high-dimensional spaces, especially when the feature space is vast. In a decision hyperplane, SVM separates the objects belonging to different classes and it tries to maximize the margin between them with support vector's help. This helps improve generalization and reduces overfitting, which is crucial for image classification..
3. **XGBoost**: This model iteratively adds trees to the model, each one attempting to correct the errors made by the previous ones. It is particularly effective for tasks like animal classification, as it handles both high-dimensional feature spaces and non-linear relationships very well. This model often has high accuracy, handles missing data well.

SVM Confusion Matrix



Prediction Comparison:

ML Model	Prediction
XGBoost	XGBoostClassifier Model took 565.85 seconds Train log_loss: 0.5905704561702899 Test log_loss: 1.46951403983652 Train Accuracy: 0.9374090247452693 Test Accuracy: 0.500823723228995
Support Vector Machines (SVM)	SVM Classifier Model took 54.60 seconds Train log_loss: 0.35967460106778165 Train_QA log_loss: 1.3544915953536454 Train Accuracy: 0.9155749636098981 Train_QA Accuracy: 0.5024711696869851
Random Forests	RandomForests Model took 10.36 seconds Train log_loss: 0.33436857311636325 Test log_loss: 1.6941756805442918 Train Accuracy: 1.0 Test Accuracy: 0.47775947281713343

Comparing the performance of the three models—Random Forests, SVM Classifier, and XGBoost, **SVM Model** has given the best performance.

- **Random Forests** is the fastest model but suffers from overfitting, as evidenced by the 100% training accuracy and poor test accuracy. It struggles to generalize well to unseen data, with a high test log loss.
- **SVM Classifier** shows good performance in terms of train accuracy (91.56%) and train log loss (0.3597). However, it is slower to train and may need further tuning for QA predictions, as train_QA accuracy is lower.
- **XGBoost** gives the best test accuracy (50.08%) but comes at the cost of a significantly longer training time and a higher train log loss (0.5906) compared to the SVM. It generalizes better than the other models, though it still could benefit from additional fine-tuning.

Future work:

Models utilizing Convolutional Neural Networks (CNNs) implemented with libraries such as Keras and TensorFlow initially demonstrated promising results. However, due to time constraints, the comprehensive development, fine-tuning, and in-depth analysis of these models could not be fully completed within the available timeframe.